```
    var token = new TokenStore.Token(expiry, subject);

    var scope = request.queryParams("scope");    #A
    if (scope != null) {     #A
        token.attributes.put("scope", scope);    #A
    }

    var tokenId = tokenStore.create(request, token);

    response.status(201);
    return new JSONObject()
            .put("token", tokenId);
}
```

#A If a scoped token is requested then store the scope in the token attributes.

To enforce the scope restrictions on a token, you can add a new access control filter that ensures that the token used to authorize a request to the API has the required scope for the operation being performed. This filter looks a lot like the existing permission filter that you added in chapter 3 and is shown in listing 7.2. (I'll discuss the differences between scopes and permissions in the next section.) To verify the scope, you need to perform several checks:

- First, check if the HTTP method of the request matches the method that this rule is for, so that you don't apply a scope for a POST request to a DELETE request or vice-versa. This is needed because Spark's filters are matched only by the path and not the request method.
- You can then look up the scope associated with the token that authorized the current request from the